

LAMBDA EXPRESSIONS AND INTRODUCTION TO RECURSION

Solutions

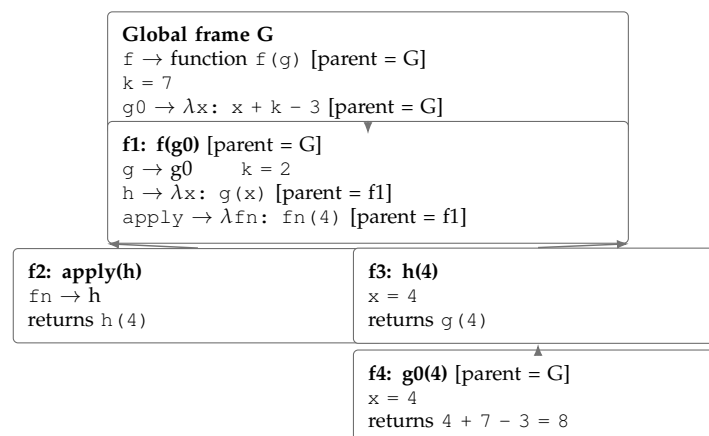
COMPUTER SCIENCE MENTORS 61A

September 14, 2026 – September 18, 2026

1 Lambda Expressions

1. Draw the environment diagram that results from running the code.

```
def f(g):  
    k = 2  
    h = lambda x: g(x)  
    apply = lambda fn: fn(4)  
    return apply(h)  
  
k = 7  
f(lambda x: x + k - 3)
```



The final value is 8. The lambda passed into f was created in the global frame, so when it evaluates k, it uses the global k = 7, not the local k = 2.

2. What would Python display?

- (a) (lambda f: f(f))(lambda x: 6) 6
- (b) (lambda f, g: g(f))(add, lambda op: op(2, 7)) 9

3. Make a lambda function, `make_multiple_checker()`, that takes in an integer `divisor`, and returns a function that takes in a value `x` and checks whether `x` is a multiple of `divisor`.

```
>>> make_multiple_checker = _____
>>> multiple_of_4 = make_multiple_checker(4)
>>> multiple_of_4(20)
True
>>> multiple_of_4(18)
False

>>> make_multiple_checker = lambda divisor: lambda x: x % divisor == 0
>>> multiple_of_4 = make_multiple_checker(4)
>>> multiple_of_4(20)
True
>>> multiple_of_4(18)
False
```

The outer lambda “remembers” `divisor`. After `make_multiple_checker(4)`, the returned function checks whether its input has remainder 0 when divided by 4.

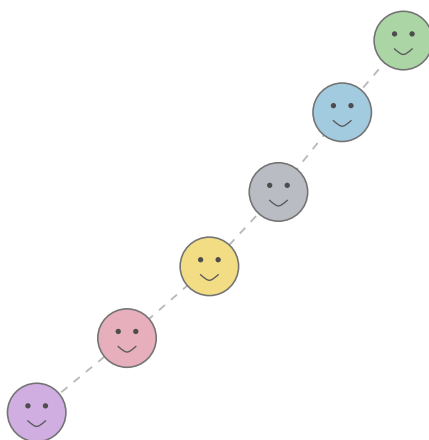
2 Introduction to Recursion

There are three steps to writing a recursive function:

1. Create base case(s)
2. Reduce your problem to a smaller subproblem and call your function recursively to solve the smaller subproblem(s)
3. Use the subproblems' solutions as pieces to construct a larger problem's solution

Real World Analogy for Recursion

Imagine that you're waiting in a long line for a campus shuttle and you want to know what position you're in.



Each person asks the person ahead the same smaller question. Eventually the question reaches the first person, where the answer is immediate: 0 people are in front. That is the **base case**. On the way back, each person adds 1, which constructs the solution to the larger problem.

When solving
recursive call

4. Complete `shuttle_line`, which takes an integer made of 0s and 2s and recursively returns the number of 2s.

```
def shuttle_line(n):  
    """Takes in an integer of 0s and 2s and returns the number of 2s.  
  
    >>> shuttle_line(0)  
    0  
    >>> shuttle_line(2)  
    1  
    >>> shuttle_line(20202)  
    3  
    >>> shuttle_line(20002)  
    2  
    """  
    if _____ :                # base case  
        return _____  
    else:  
        return _____ + _____ # recursive case  
  
if n == 0:  
    return 0  
else:  
    return (n % 10 == 2) + shuttle_line(n // 10)
```

The recursive call removes one digit using `n // 10`. The expression `n % 10 == 2` contributes 1 when the last digit is 2 and 0 otherwise. The base case `n == 0` stops the recursion.

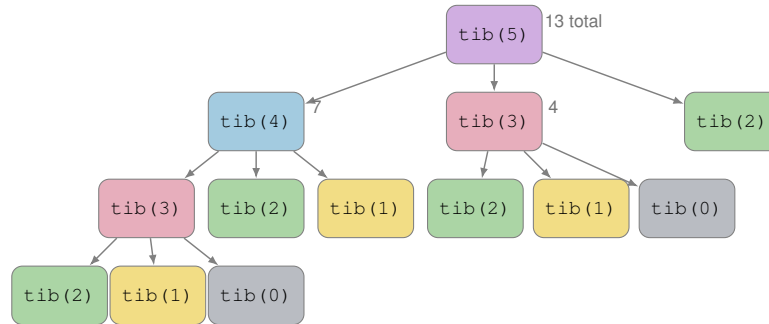
5. The *Tibonacci sequence* begins $t_0 = 1$, $t_1 = 2$, $t_2 = 4$, and for $n \geq 3$, t_n is the sum of the previous three terms. Complete `tib(n)` and identify the three parts of recursive design.

```
def tib(n):  
    """  
    >>> tib(0)  
    1  
    >>> tib(1)  
    2  
    >>> tib(2)  
    4  
    >>> tib(3)  
    7  
    >>> tib(4)  
    13  
    """  
    if _____:  
        return _____  
    return _____  
  
if n <= 2:  
    return 2 ** n  
return tib(n - 1) + tib(n - 2) + tib(n - 3)
```

- Base cases: $n \leq 2$, returning 1, 2, 4 as $2 ** n$.
- Smaller subproblems: `tib(n - 1)`, `tib(n - 2)`, `tib(n - 3)`.
- Construct the larger answer by adding the three recursive results.

6. Including the original call, how many calls are made to `tib` when you evaluate `tib(5)`? **13 calls.**

One way to see it is to draw the full call tree. This tree is newly drawn for this worksheet; every non-base call has exactly three children.



$$C(0) = C(1) = C(2) = 1$$

$$C(3) = 1 + C(2) + C(1) + C(0) = 4$$

$$C(4) = 1 + C(3) + C(2) + C(1) = 7$$

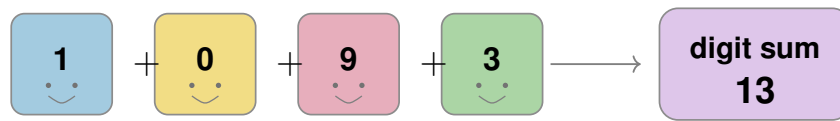
$$C(5) = 1 + C(4) + C(3) + C(2) = 13$$

The recurrence for the number of calls is the same shape as the function itself, except each call also counts the current call with the leading 1.

7. Complete `sum_digits`, which takes in a nonnegative integer `n` and recursively returns the sum of its digits.

```
def sum_digits(n):  
    """Takes in a nonnegative integer and returns the sum of its digits.  
  
    >>> sum_digits(0)  
    0  
    >>> sum_digits(5)  
    5  
    >>> sum_digits(71)  
    8  
    >>> sum_digits(1093)  
    13  
    """  
  
    if n < 10:  
        return n  
    return n % 10 + sum_digits(n // 10)
```

A one-digit number is already its own digit sum. Otherwise, add the last digit to the digit sum of the remaining prefix.



A tiny digit parade: peel off one digit at a time.

8. Implement a recursive version of popzap.

```
def popzap(n):  
    """Prints the numbers from 1 to n. Multiples of 4 print 'pop',  
    multiples of 6 print 'zap', and multiples of both print 'popzap'.  
    You must do this recursively!  
    """  
  
    if n == 0:  
        return  
    popzap(n - 1)  
    if n % 12 == 0:  
        print('popzap')  
    elif n % 4 == 0:  
        print('pop')  
    elif n % 6 == 0:  
        print('zap')  
    else:  
        print(n)
```

The recursive call comes *before* the printing logic, so the smaller call prints 1 through $n - 1$ first. Then the current call prints the correct representation of n . Checking 12 first handles numbers divisible by both 4 and 6.



The recursion does the counting; the current call chooses the word.